Complete Function Flow Guide - News App

> *A detailed guide explaining exactly which functions run when, in what order, and what each function does.*

Table of Contents

---

1. App Startup Flow

What Happens When You Click the App Icon?

```
┌─────────────────────────────────────────────────────────────────────┐
│                        APP STARTUP SEQUENCE                         │
├─────────────────────────────────────────────────────────────────────┤
│                                                                     │
│  STEP 1: Android System                                             │
│  ──────────────────────                                             │
│  User taps app icon → Android reads AndroidManifest.xml             │
│  → Finds: android:name=".NewsApp" (Application class)               │
│  → Finds: MainActivity with LAUNCHER intent filter                  │
│                                                                     │
│  STEP 2: NewsApp.kt (Application Class)                             │
│  ──────────────────────────────────────                             │
│  @HiltAndroidApp triggers Hilt initialization                       │
│  → Creates dependency injection container                           │
│  → All @Singleton objects are prepared (but not created yet)        │
│                                                                     │
│  STEP 3: MainActivity.kt                                            │
│  ────────────────────────                                           │
│  onCreate() is called by Android                                    │
│  → @AndroidEntryPoint enables injection in this Activity            │
│  → setContent { } starts Jetpack Compose                            │
│                                                                     │
│  STEP 4: Compose UI Setup                                           │
│  ────────────────────────                                           │
│  NewsAppTheme { } → Applies Material3 colors and typography         │
│  Surface { } → Creates background container                         │
│  rememberNavController() → Creates navigation controller            │
│  NavGraph() → Sets up all screen routes                             │
│                                                                     │
│  STEP 5: Navigation to First Screen                                 │
│  ──────────────────────────────────                                 │
│  NavHost reads: startDestination = Screen.LoginScreen.route         │
│  → Navigates to LoginScreen composable                              │
│  → LoginScreen is displayed to user                                 │
│                                                                     │
└─────────────────────────────────────────────────────────────────────┘
```

Detailed Function Calls (In Order)

Step	File	Function/Code	What It Does
1	`AndroidManifest.xml`	System reads manifest	Tells Android which Application class and Activity to launch
2	`NewsApp.kt`	`@HiltAndroidApp` annotation	Initializes Hilt dependency injection framework
3	`MainActivity.kt`	`onCreate(savedInstanceState)`	Android lifecycle callback - Activity is being created
4	`MainActivity.kt`	`setContent { }`	Tells Compose to render UI inside this Activity
5	`Theme.kt`	`NewsAppTheme { }`	Applies app-wide colors, typography, shapes
6	`MainActivity.kt`	`Surface(...)`	Creates a container with background color
7	`MainActivity.kt`	`rememberNavController()`	Creates NavController to manage screen navigation
8	`NavGraph.kt`	`NavGraph(navController)`	Composable function that defines all routes
9	`NavGraph.kt`	`NavHost(startDestination = "login_screen")`	Container that displays screens based on current route
10	`LoginScreen.kt`	`LoginScreen(navController)`	First screen shown to user
11	`LoginScreen.kt`	`hiltViewModel<LoginViewModel>()`	Hilt creates LoginViewModel instance
12	`LoginViewModel.kt`	`LoginViewModel @Inject constructor()`	ViewModel is initialized with empty state

Code Walkthrough

Step 1: NewsApp.kt

```
@HiltAndroidApp  // ← This annotation does the magic
class NewsApp : Application()
```

What happens: When Android starts the app, it sees `@HiltAndroidApp` and:

1. Creates a Hilt component (container for dependencies)
2. Sets up the dependency graph
3. Makes all `@Inject` annotations work throughout the app

Step 2: MainActivity.kt

```
@AndroidEntryPoint  // ← Allows Hilt to inject into this Activity
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {                          // ← Start Compose UI
            NewsAppTheme {                    // ← Apply theme
                Surface(...) {
                    val navController = rememberNavController()  // ← Create navigator
                    NavGraph(navController = navController)       // ← Setup routes
                }
            }
        }
```

```
    }
}
```

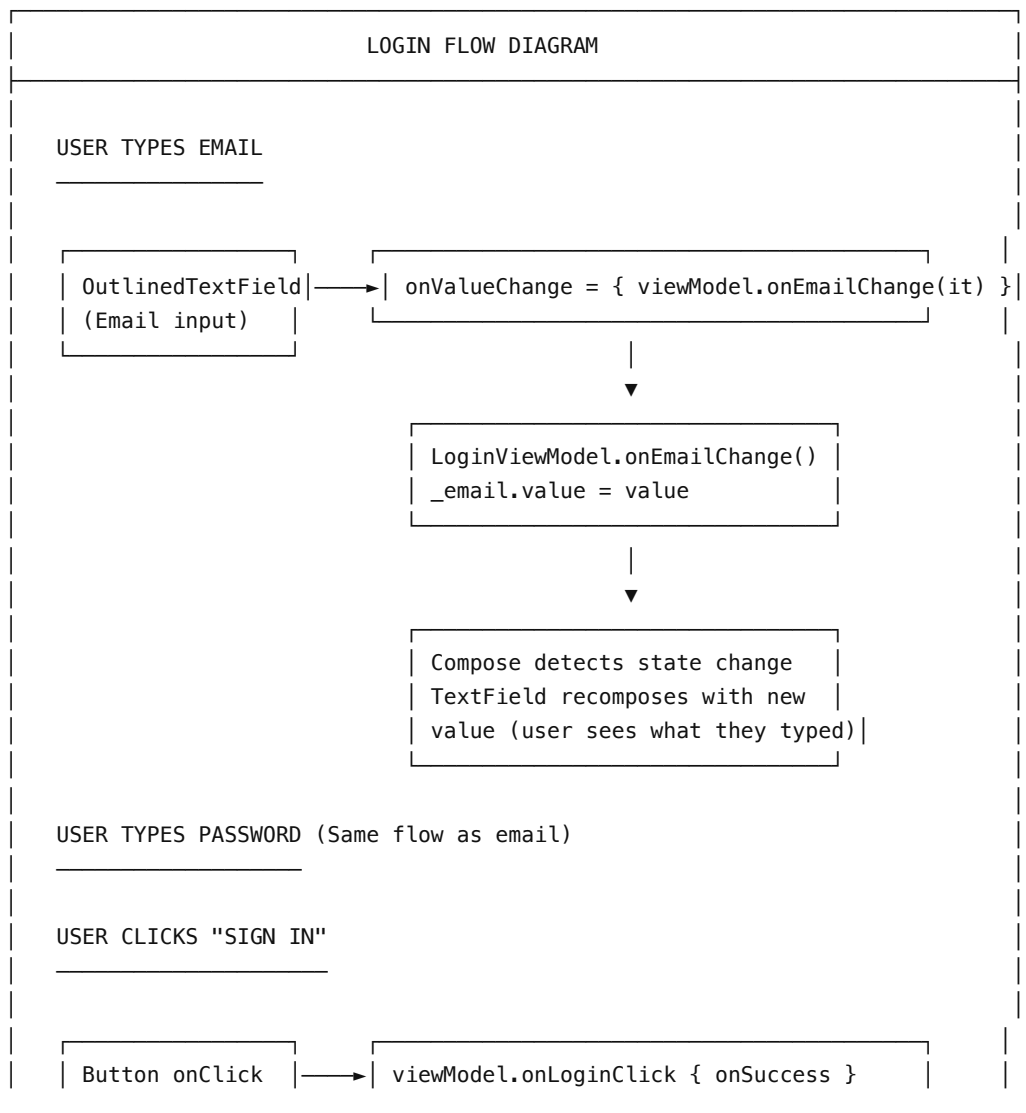
Step 3: NavGraph.kt

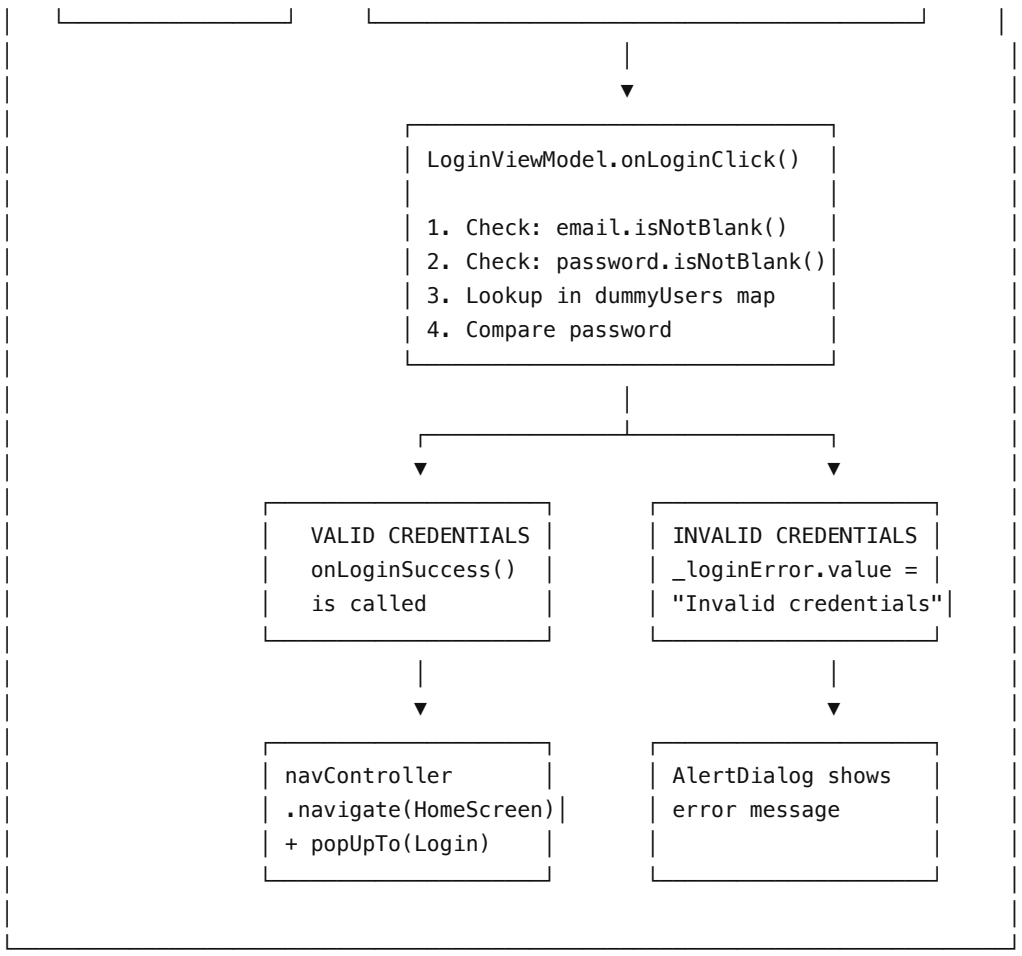
```
@Composable
fun NavGraph(navController: NavHostController) {
    NavHost(
        navController = navController,
        startDestination = Screen.LoginScreen.route  // ← "login_screen"
    ) {
        composable(route = Screen.LoginScreen.route) {
            LoginScreen(navController = navController)  // ← First screen shown
        }
        // ... other screens
    }
}
```

---

2. Login Flow

What Happens When You Enter Email and Password?





Login Validation Code Explained

File: `LoginViewModel.kt`

```
// STEP 1: State variables hold the current values
private val _email = mutableStateOf("")        // Holds email text
private val _password = mutableStateOf("")     // Holds password text
private val _loginError = mutableStateOf<String?>(null)  // Holds error message

// STEP 2: Hardcoded users for demo (in real app, this would be API call)
private val dummyUsers = mapOf(
    "test@example.com" to "password",
    "parag@icici.com" to "password",
    // ... more users
)

// STEP 3: Called when user types in email field
fun onEmailChange(value: String) {
    _email.value = value  // Updates state → triggers UI recomposition
}

// STEP 4: Called when user types in password field
fun onPasswordChange(value: String) {
    _password.value = value
}

// STEP 5: Called when user clicks "Sign In" button
```

```
fun onLoginClick(onLoginSuccess: () -> Unit) {

    // Validation Step 1: Check if fields are not empty
    if (_email.value.isNotBlank() && _password.value.isNotBlank()) {

        // Validation Step 2: Convert email to lowercase for comparison
        val enteredEmail = _email.value.lowercase()

        // Validation Step 3: Check if email exists AND password matches
        if (dummyUsers.containsKey(enteredEmail) &&
            dummyUsers[enteredEmail] == _password.value) {

            // SUCCESS: Clear any error and call success callback
            _loginError.value = null
            onLoginSuccess()  // This triggers navigation to HomeScreen

        } else {
            // FAILURE: Invalid credentials
            _loginError.value = "Invalid credentials. Please try again."
        }
    } else {
        // FAILURE: Empty fields
        _loginError.value = "Email and password cannot be empty."
    }
}
```

Login Screen UI Code Explained

File: `LoginScreen.kt`

```
@Composable
fun LoginScreen(
    navController: NavController,
    viewModel: LoginViewModel = hiltViewModel()  // ← Hilt provides ViewModel
) {
    // OBSERVE STATE: These values update automatically when ViewModel changes
    val email by viewModel.email        // Current email text
    val password by viewModel.password  // Current password text
    val loginError by viewModel.loginError  // Error message (null if no error)

    // EMAIL TEXT FIELD
    OutlinedTextField(
        value = email,  // ← Displays current email from ViewModel
        onValueChange = { viewModel.onEmailChange(it) },  // ← Updates ViewModel
        label = { Text("Email") }
    )

    // PASSWORD TEXT FIELD
    OutlinedTextField(
        value = password,
        onValueChange = { viewModel.onPasswordChange(it) },
        label = { Text("Password") },
        visualTransformation = PasswordVisualTransformation()  // ← Hides text
    )

    // SIGN IN BUTTON
```

```
    Button(
        onClick = {
            viewModel.onLoginClick {
                // This lambda is called on success
                navController.navigate(Screen.HomeScreen.route) {
                    // Remove LoginScreen from back stack so user can't go back
                    popUpTo(Screen.LoginScreen.route) { inclusive = true }
                }
            }
        }
    ) {
        Text("Sign In")
    }

    // ERROR DIALOG (shown only when loginError is not null)
    loginError?.let { error ->
        AlertDialog(
            onDismissRequest = { viewModel.clearError() },
            title = { Text("Login Failed") },
            text = { Text(error) },
            confirmButton = {
                Button(onClick = { viewModel.clearError() }) {
                    Text("OK")
                }
            }
        )
    }
}
```

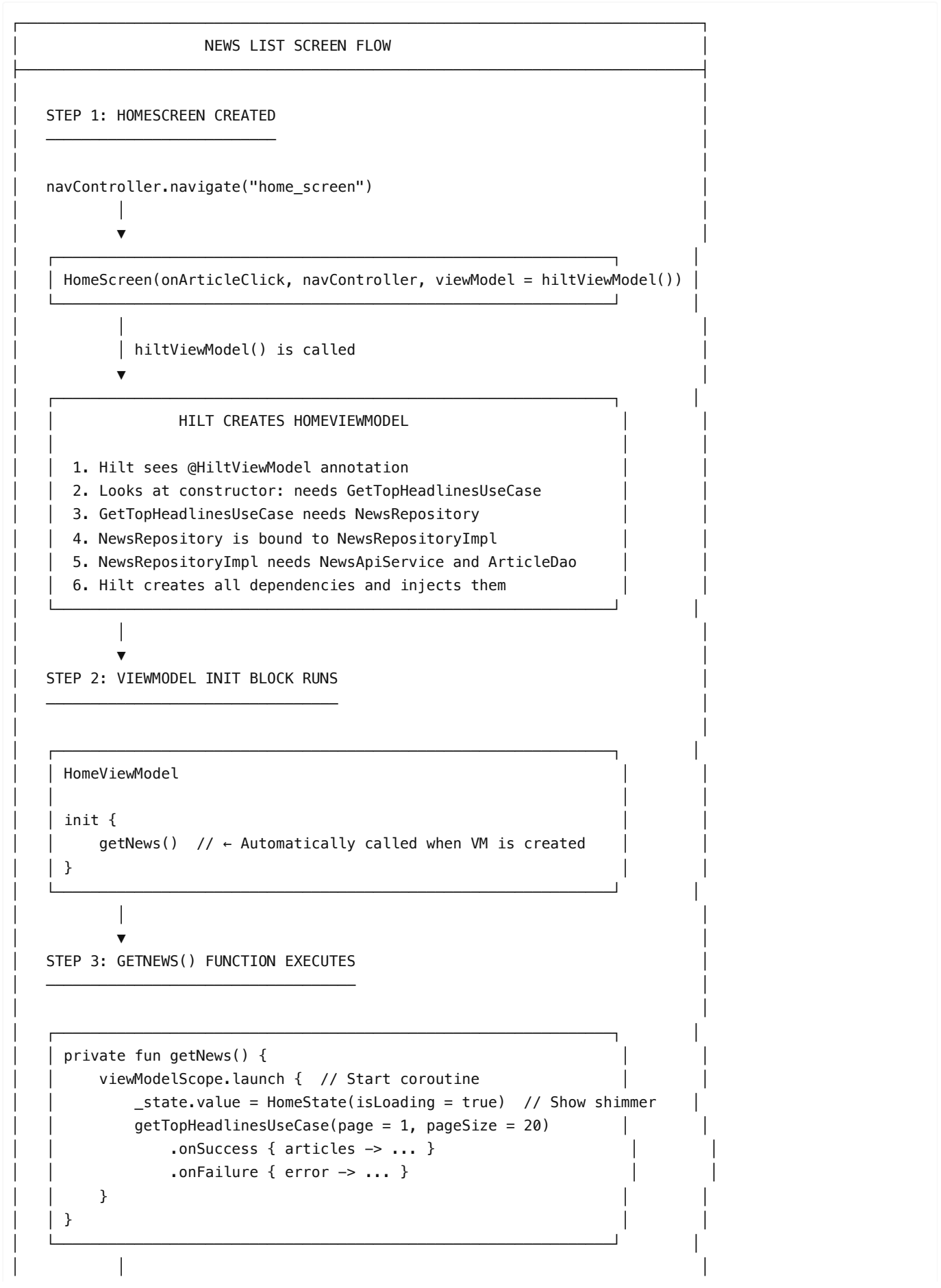
Function Call Sequence for Login

Step	Trigger	Function	File	What Happens
1	User types "t"	`onValueChange` callback	`LoginScreen.kt:262`	Calls `viewModel.onEmailChange("t")`
2	-	`onEmailChange("t")`	`LoginViewModel.kt:53`	`_email.value = "t"`
3	State change	Compose recomposition	`LoginScreen.kt:260`	TextField shows "t"
4	User finishes email	-	-	`_email.value = "test@example.com"`
5	User types password	`onPasswordChange()`	`LoginViewModel.kt:61`	`_password.value = "password"`
6	User clicks Sign In	`onClick` callback	`LoginScreen.kt:316`	Calls `viewModel.onLoginClick { ... }`
7	-	`onLoginClick()`	`LoginViewModel.kt:70`	Validates credentials
8a	Valid	`onLoginSuccess()`	`LoginScreen.kt:318`	`navController.navigate("home_screen")`
8b	Invalid	-	`LoginViewModel.kt:83`	`_loginError.value = "Invalid..."`
9b	State change	Compose recomposition	`LoginScreen.kt:402`	AlertDialog appears

---

3. News List Screen Flow

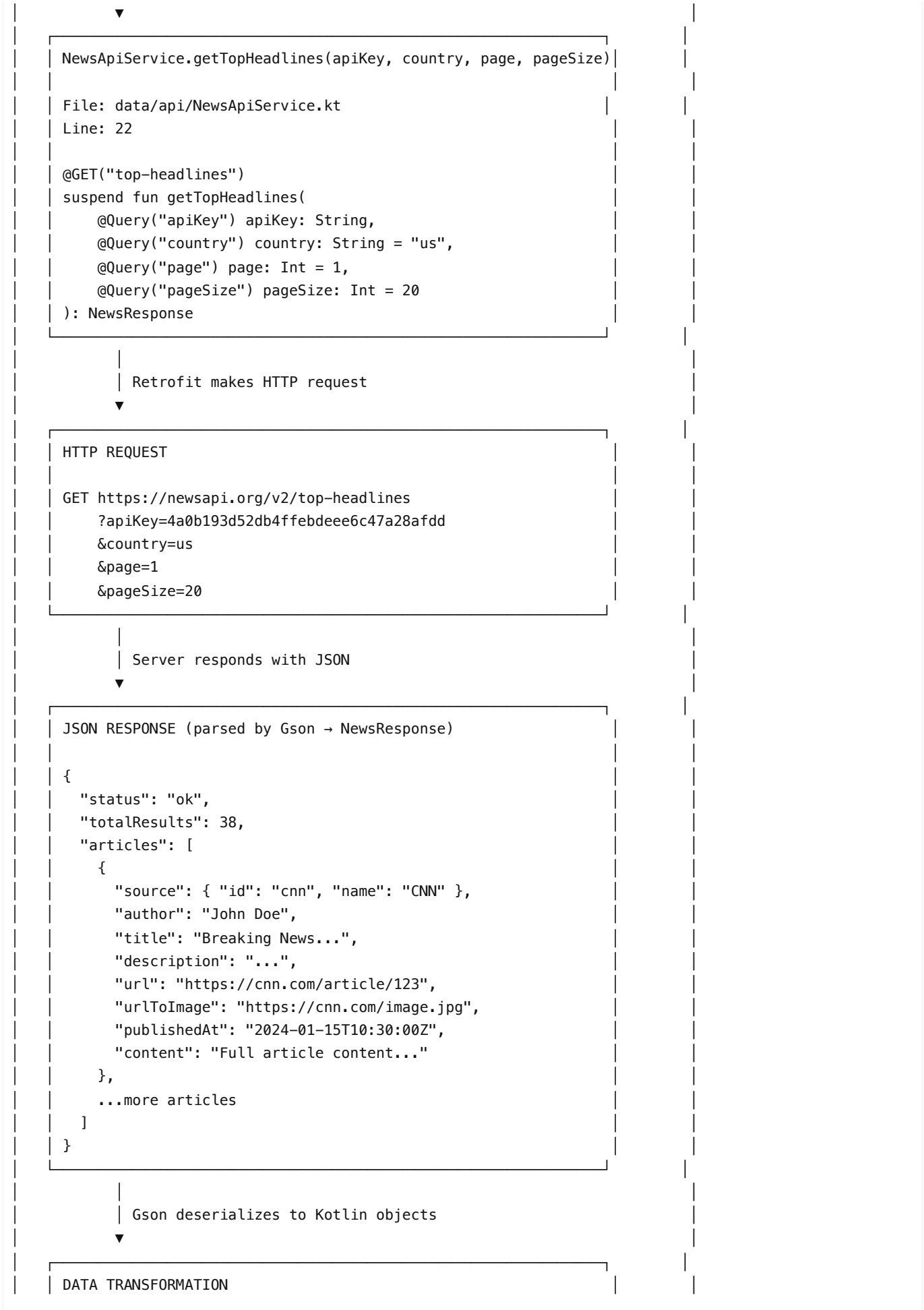
What Happens When HomeScreen Loads?

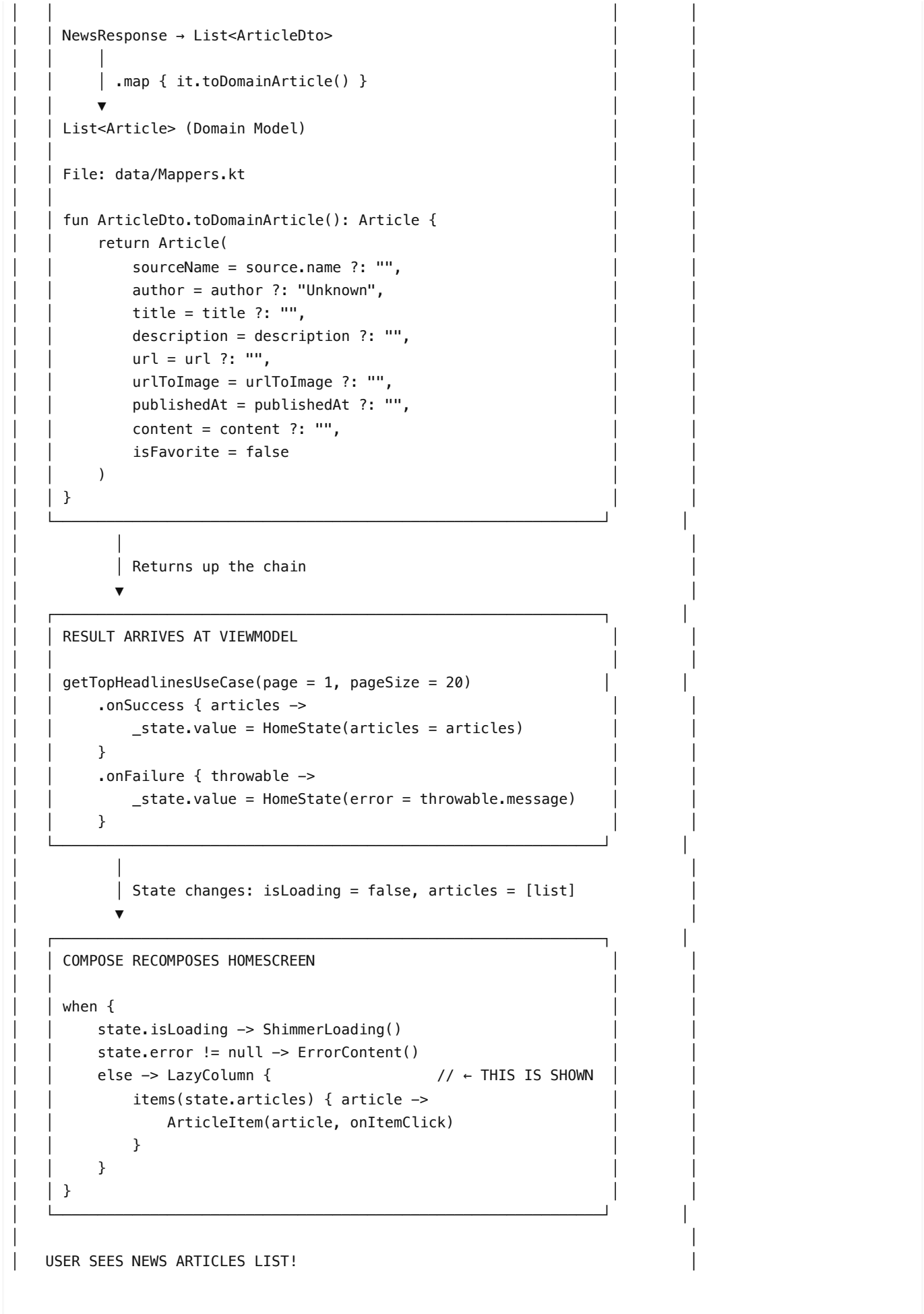


```
│           │ State changes to isLoading = true                                │
│           ▼                                                                  │
│    ┌──────────────────────────────────────────────────────────────┐          │
│    │ HomeScreen UI recomposes                                     │          │
│    │                                                              │          │
│    │ when {                                                       │          │
│    │     state.isLoading -> ShimmerLoading()  // ← THIS IS SHOWN  │          │
│    │     state.error != null -> ErrorContent()                    │          │
│    │     else -> LazyColumn(articles)                             │          │
│    │ }                                                            │          │
│    └──────────────────────────────────────────────────────────────┘          │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘
```

API Call Chain (Complete Data Flow)

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                        DATA FETCHING FLOW                                    │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│   HomeViewModel.getNews()                                                    │
│           │                                                                  │
│           │ calls                                                            │
│           ▼                                                                  │
│    ┌──────────────────────────────────────────────────────────────┐          │
│    │ GetTopHeadlinesUseCase.invoke(page = 1, pageSize = 20)       │          │
│    │                                                              │          │
│    │ File: domain/usecase/NewsUseCases.kt                         │          │
│    │ Line: 57                                                     │          │
│    │                                                              │          │
│    │ suspend operator fun invoke(page, pageSize): Result<List<Article>> {   │
│    │     return runCatching {                                     │          │
│    │         newsRepository.getTopHeadlines(page, pageSize)       │          │
│    │     }                                                        │          │
│    │ }                                                            │          │
│    └──────────────────────────────────────────────────────────────┘          │
│           │                                                                  │
│           │ calls                                                            │
│           ▼                                                                  │
│    ┌──────────────────────────────────────────────────────────────┐          │
│    │ NewsRepositoryImpl.getTopHeadlines(page, pageSize)           │          │
│    │                                                              │          │
│    │ File: data/repository/NewsRepositoryImpl.kt                  │          │
│    │ Line: 76                                                     │          │
│    │                                                              │          │
│    │ override suspend fun getTopHeadlines(page, pageSize): List<Article> {  │
│    │     val response = apiService.getTopHeadlines(               │          │
│    │         apiKey = API_KEY,                                    │          │
│    │         page = page,                                         │          │
│    │         pageSize = pageSize                                  │          │
│    │     )                                                        │          │
│    │     return response.articles.map { it.toDomainArticle() }    │          │
│    │ }                                                            │          │
│    └──────────────────────────────────────────────────────────────┘          │
│           │                                                                  │
│           │ calls                                                            │
```


NewsApiService.getTopHeadlines(apiKey, country, page, pageSize)
File: data/api/NewsApiService.kt
Line: 22
@GET("top-headlines")
suspend fun getTopHeadlines(
@Query("apiKey") apiKey: String,
@Query("country") country: String = "us",
@Query("page") page: Int = 1,
@Query("pageSize") pageSize: Int = 20
): NewsResponse
Retrofit makes HTTP request
HTTP REQUEST
GET https://newsapi.org/v2/top-headlines
?apiKey=4a0b193d52db4ffebdeee6c47a28afdd
&country=us
&page=1
&pageSize=20
Server responds with JSON
JSON RESPONSE (parsed by Gson → NewsResponse)
{
"status": "ok",
"totalResults": 38,
"articles": [
{
"source": { "id": "cnn", "name": "CNN" },
"author": "John Doe",
"title": "Breaking News...",
"description": "...",
"url": "https://cnn.com/article/123",
"urlToImage": "https://cnn.com/image.jpg",
"publishedAt": "2024-01-15T10:30:00Z",
"content": "Full article content..."
},
...more articles
]
}
Gson deserializes to Kotlin objects
DATA TRANSFORMATION


NewsResponse → List<ArticleDto>
.map { it.toDomainArticle() }
List<Article> (Domain Model)
File: data/Mappers.kt
fun ArticleDto.toDomainArticle(): Article {
return Article(
sourceName = source.name ?: "",
author = author ?: "Unknown",
title = title ?: "",
description = description ?: "",
url = url ?: "",
urlToImage = urlToImage ?: "",
publishedAt = publishedAt ?: "",
content = content ?: "",
isFavorite = false
)
}
Returns up the chain
RESULT ARRIVES AT VIEWMODEL
getTopHeadlinesUseCase(page = 1, pageSize = 20)
.onSuccess { articles ->
_state.value = HomeState(articles = articles)
}
.onFailure { throwable ->
_state.value = HomeState(error = throwable.message)
}
State changes: isLoading = false, articles = [list]
COMPOSE RECOMPOSES HOMESCREEN
when {
state.isLoading -> ShimmerLoading()
state.error != null -> ErrorContent()
else -> LazyColumn { // ← THIS IS SHOWN
items(state.articles) { article ->
ArticleItem(article, onItemClick)
}
}
}
USER SEES NEWS ARTICLES LIST!

```
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Complete Function Call Sequence

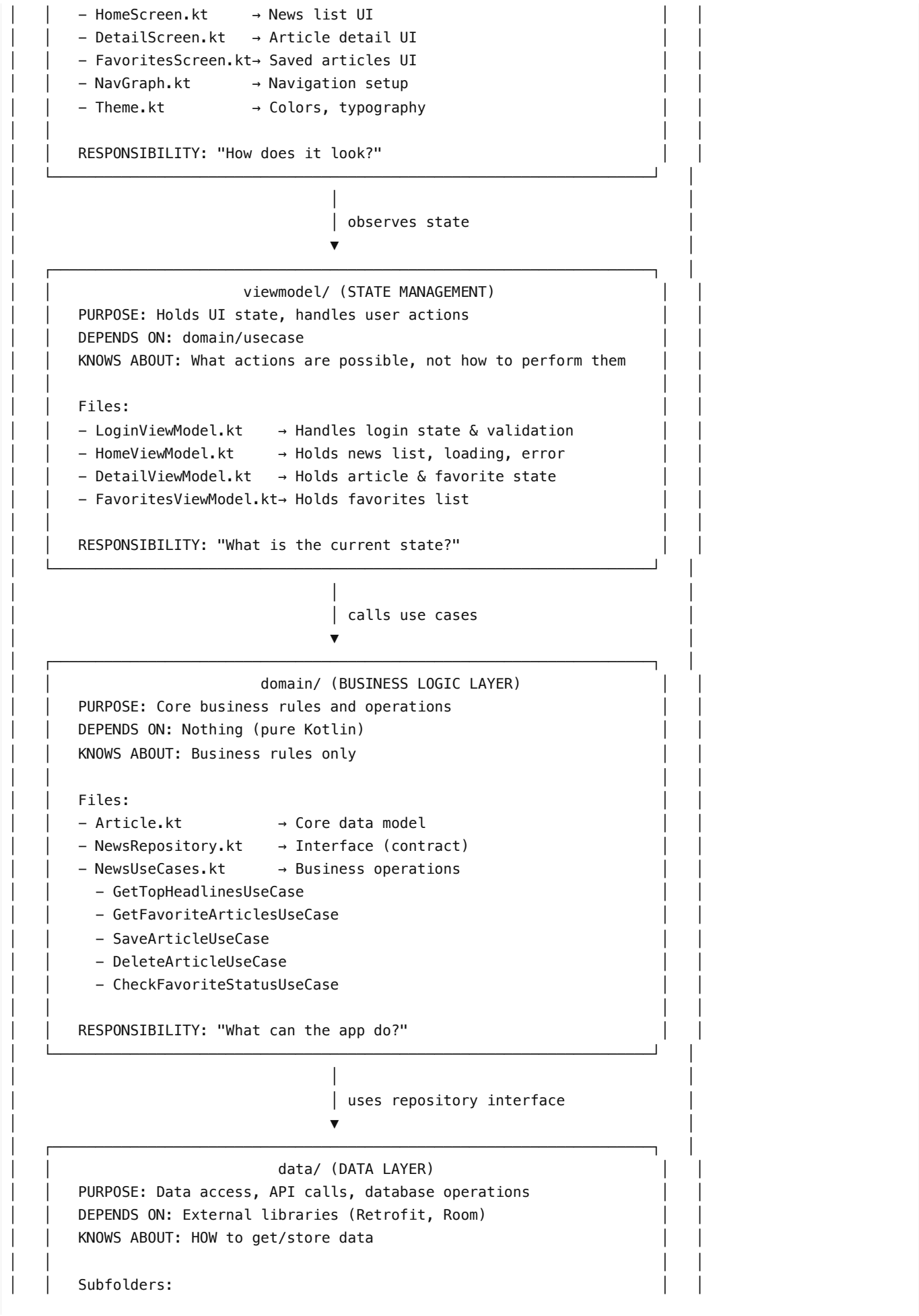
Step	Function	File:Line	What Happens
1	`HomeScreen()`	`HomeScreen.kt:51`	Composable function called
2	`hiltViewModel<HomeViewModel>()`	`HomeScreen.kt:54`	Hilt creates ViewModel
3	`HomeViewModel constructor`	`HomeViewModel.kt:61`	ViewModel initialized
4	`init { getNews() }`	`HomeViewModel.kt:89`	init block runs getNews()
5	`viewModelScope.launch {}`	`HomeViewModel.kt:113`	Starts coroutine
6	`_state.value = HomeState(isLoading = true)`	`HomeViewModel.kt:115`	Set loading state
7	UI Recomposition	`HomeScreen.kt:153`	ShimmerLoading() shown
8	`getTopHeadlinesUseCase(1, 20)`	`HomeViewModel.kt:119`	Call use case
9	`runCatching { newsRepository.getTopHeadlines() }`	`NewsUseCases.kt:62`	Use case calls repository
10	`apiService.getTopHeadlines()`	`NewsRepositoryImpl.kt:78`	Repository calls API
11	HTTP GET Request	Retrofit	Network call to newsapi.org
12	JSON → NewsResponse	Gson	Deserialize response
13	`response.articles.map { it.toDomainArticle() }`	`NewsRepositoryImpl.kt:85`	Convert DTOs to domain models
14	`Result.success(articles)`	`NewsUseCases.kt:62`	Wrap in Result
15	`.onSuccess { articles -> }`	`HomeViewModel.kt:120`	Handle success
16	`_state.value = HomeState(articles = articles)`	`HomeViewModel.kt:122`	Update state
17	UI Recomposition	`HomeScreen.kt:166`	LazyColumn shown with articles

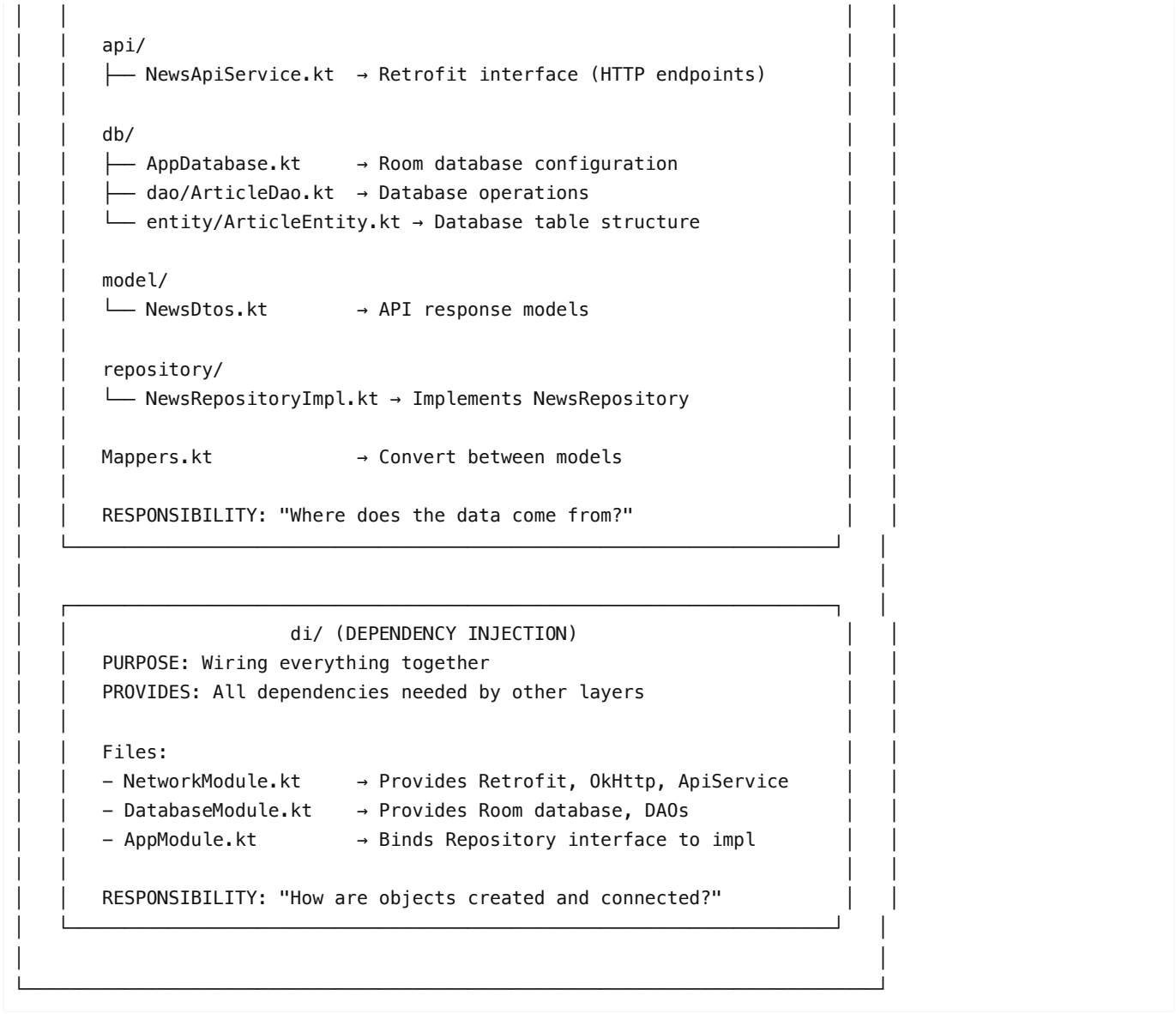
---

4. Folder Structure Deep Dive

Why This Folder Structure?

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    CLEAN ARCHITECTURE FOLDER STRUCTURE                   │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│   The folders are organized by LAYER, not by FEATURE.                    │
│   This separates WHAT the app does from HOW it does it.                  │
│                                                                          │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │                        ui/ (PRESENTATION LAYER)                  │   │
│   │   PURPOSE: Everything the user sees and interacts with           │   │
│   │   DEPENDS ON: viewmodel/, domain/                                │   │
│   │   KNOWS ABOUT: How to display data, not where it comes from      │   │
│   │                                                                  │   │
│   │   Files:                                                         │   │
│   │   - LoginScreen.kt    → Login page UI                            │   │
```


- HomeScreen.kt → News list UI
- DetailScreen.kt → Article detail UI
- FavoritesScreen.kt→ Saved articles UI
- NavGraph.kt → Navigation setup
- Theme.kt → Colors, typography
RESPONSIBILITY: "How does it look?"
observes state
viewmodel/ (STATE MANAGEMENT)
PURPOSE: Holds UI state, handles user actions
DEPENDS ON: domain/usecase
KNOWS ABOUT: What actions are possible, not how to perform them
Files:
- LoginViewModel.kt → Handles login state & validation
- HomeViewModel.kt → Holds news list, loading, error
- DetailViewModel.kt → Holds article & favorite state
- FavoritesViewModel.kt→ Holds favorites list
RESPONSIBILITY: "What is the current state?"
calls use cases
domain/ (BUSINESS LOGIC LAYER)
PURPOSE: Core business rules and operations
DEPENDS ON: Nothing (pure Kotlin)
KNOWS ABOUT: Business rules only
Files:
- Article.kt → Core data model
- NewsRepository.kt → Interface (contract)
- NewsUseCases.kt → Business operations
- GetTopHeadlinesUseCase
- GetFavoriteArticlesUseCase
- SaveArticleUseCase
- DeleteArticleUseCase
- CheckFavoriteStatusUseCase
RESPONSIBILITY: "What can the app do?"
uses repository interface
data/ (DATA LAYER)
PURPOSE: Data access, API calls, database operations
DEPENDS ON: External libraries (Retrofit, Room)
KNOWS ABOUT: HOW to get/store data
Subfolders:



Each Folder's Files Explained

`ui/` - User Interface

File	Purpose	Key Functions
`LoginScreen.kt`	Login page with email/password	`LoginScreen()`, `FloatingIconAnimation()`
`HomeScreen.kt`	News list display	`HomeScreen()`, `ErrorContent()`
`DetailScreen.kt`	Single article view	`DetailScreen()`
`FavoritesScreen.kt`	Saved articles list	`FavoritesScreen()`, `EmptyFavoritesContent()`
`NavGraph.kt`	Navigation routes	`NavGraph()`
`Screen.kt`	Route definitions	`Screen.LoginScreen`, `Screen.HomeScreen`, etc.
`ArticleItem.kt`	Article card component	`ArticleItem()`
`Shimmer.kt`	Loading animation	`ShimmerLoading()`, `ShimmerArticleItem()`
`Theme.kt`	App theme	`NewsAppTheme()`

`Color.kt`	Color definitions	`Purple80`, `Pink40`, etc.
`Type.kt`	Typography	`Typography`

`viewmodel/` - State Management

File	State Class	Key Functions
`LoginViewModel.kt`	`email`, `password`, `loginError`	`onEmailChange()`, `onPasswordChange()`, `onLoginClick()`, `clearError()`
`HomeViewModel.kt`	`HomeState(isLoading, articles, error)`	`getNews()`
`DetailViewModel.kt`	`DetailState(isFavorite)`	`initArticle()`, `onFavoriteClick()`, `observeFavoriteStatus()`
`FavoritesViewModel.kt`	`FavoritesState(articles)`	`getFavoriteArticles()`

`domain/` - Business Logic

File	Purpose	Contents
`Article.kt`	Core business model	`data class Article(...)`
`NewsRepository.kt`	Data access contract	`interface NewsRepository { getTopHeadlines(), getFavoriteArticles(), ... }`
`NewsUseCases.kt`	Business operations	5 use case classes

`data/` - Data Access

File	Purpose	Contents
`NewsApiService.kt`	HTTP endpoints	`@GET("top-headlines") suspend fun getTopHeadlines(...)`
`NewsDtos.kt`	API models	`NewsResponse`, `ArticleDto`, `SourceDto`
`NewsRepositoryImpl.kt`	Repository implementation	Coordinates API and Database
`AppDatabase.kt`	Room setup	`@Database abstract class AppDatabase`
`ArticleDao.kt`	DB operations	`@Insert`, `@Delete`, `@Query` functions
`ArticleEntity.kt`	DB table	`@Entity data class ArticleEntity`
`Mappers.kt`	Model conversion	`toDomainArticle()`, `toArticleEntity()`

`di/` - Dependency Injection

File	Provides	How
`NetworkModule.kt`	`HttpLoggingInterceptor`, `OkHttpClient`, `Retrofit`, `NewsApiService`	`@Provides` functions
`DatabaseModule.kt`	`AppDatabase`, `ArticleDao`	`@Provides` functions
`AppModule.kt`	`NewsRepository` → `NewsRepositoryImpl`	`@Binds` function

---

5. Interview Questions & Answers

Q1: What happens when the app starts?

Answer:

1. Android reads `AndroidManifest.xml` and finds `NewsApp` as the Application class
2. `@HiltAndroidApp` initializes Hilt dependency injection
3. `MainActivity.onCreate()` is called
4. `setContent{}` starts Jetpack Compose
5. `NewsAppTheme` applies styling
6. `NavGraph` is created with `startDestination = "login_screen"`
7. `LoginScreen` composable is displayed
8. `hiltViewModel<LoginViewModel>()` creates the ViewModel

Q2: How does data flow from API to UI?

Answer:

```
API Response (JSON)
    ↓ Gson deserializes
NewsResponse (DTO)
    ↓ Repository extracts articles
List<ArticleDto>
    ↓ Mapper converts
List<Article> (Domain Model)
    ↓ Use case wraps in Result
Result<List<Article>>
    ↓ ViewModel updates state
HomeState(articles = list)
    ↓ Compose observes
UI Recomposes with new data
```

Q3: What is the role of ViewModel?

Answer:

- Holds UI state (`mutableStateOf` or `StateFlow`)
- Survives configuration changes (screen rotation)
- Handles user actions (button clicks)
- Calls use cases to perform operations
- Does NOT know about UI components (no `Context` , no `View`)

Q4: What is the role of Use Case?

Answer:

- Single responsibility: one operation per class
- Contains business logic
- Called by ViewModel
- Calls Repository
- Makes code reusable (same use case can be used by multiple ViewModels)
- Makes code testable (can test business logic without Android framework)

Q5: What is the role of Repository?

Answer:

- Single source of truth for data
- Abstracts data sources (API, Database)
- Coordinates between remote and local data
- Converts between DTOs, Entities, and Domain models
- ViewModel doesn't know if data comes from API or cache

Q6: Why use Hilt for dependency injection?

Answer:

- Reduces boilerplate code
- Compile-time verification (errors caught at build time)
- Automatic lifecycle management
- Easy testing (can swap implementations)
- Integrates with Android architecture components

Q7: How does Room database work with Flow?

Answer:

- Room can return `Flow<List<Entity>>` from queries
- When data in database changes, Room automatically emits new value
- UI observes Flow and updates without manual refresh
- Example: When you favorite an article, the favorites list updates automatically

Q8: What is the difference between DTO, Entity, and Domain Model?

Answer:

Type	Purpose	Where Used
DTO (Data Transfer Object)	Matches API response structure	`data/model/`
Entity	Matches database table structure	`data/db/entity/`
Domain Model	Core business model	`domain/model/`

Mappers convert between these to keep layers independent.

Q9: How does login validation work?

Answer:

1. User types in TextField → `onValueChange` calls `viewModel.onEmailChange()`
2. ViewModel updates `_email.value`
3. Compose recomposes TextField with new value
4. User clicks Sign In → `onClick` calls `viewModel.onLoginClick()`
5. ViewModel checks: `email.isNotBlank() && password.isNotBlank()`
6. ViewModel looks up email in `dummyUsers` map
7. If found and password matches → call `onLoginSuccess()` callback
8. If not → set `_loginError.value` → AlertDialog shows

Q10: What is the purpose of `suspend` keyword?

Answer:

- Marks a function that can be paused and resumed
- Used for long-running operations (network, database)
- Must be called from a coroutine (`viewModelScope.launch {}`)
- Prevents blocking the main thread
- Example: `suspend fun getTopHeadlines()` doesn't block UI while fetching news

Summary Table: Which Functions Run When

User Action	Functions Called (in order)

App Opens	`NewsApp.init()` → `MainActivity.onCreate()` → `setContent()` → `NavGraph()` → `LoginScreen()` → `hiltViewModel()` → `LoginViewModel.init()`
Type Email	`OutlinedTextField.onValueChange()` → `LoginViewModel.onEmailChange()` → Compose recomposes
Click Sign In	`Button.onClick()` → `LoginViewModel.onLoginClick()` → validates → `navController.navigate()`
HomeScreen Loads	`HomeScreen()` → `hiltViewModel()` → `HomeViewModel.init()` → `getNews()` → `getTopHeadlinesUseCase()` → `NewsRepositoryImpl.getTopHeadlines()` → `NewsApiService.getTopHeadlines()` → HTTP Request → Response → Mapping → State Update → UI Recomposition

---

This guide should help you understand and answer any question about this codebase!